

CS & IT ENGINEERING

DATA STRUCTURE

Tree (Part-I)



JOIN TELEGRAM TO ENROLL

SHAIENDRA SIR

Tree and Binary Tree

- ✓• What is Tree?
- ✓• Important Terminologies in Tree
- ✓• N-ary Tree
- ✓• Binary Tree
- ✓• Properties of Binary Tree

Implement Queue using Stack :-

$$\underline{n+m} \leq x \leq \underline{2n}, \quad \underline{2m} \leq y \leq \underline{n+m}$$

$n = 5 \Rightarrow \text{Enqueue}$

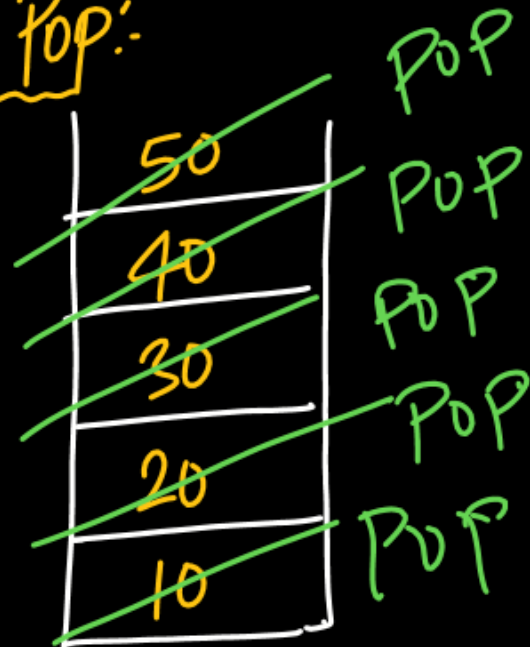
$m = 3 \Rightarrow \text{Dequeue}$

$$x_{\min} = 8 \quad | \quad x_{\max} = 10$$

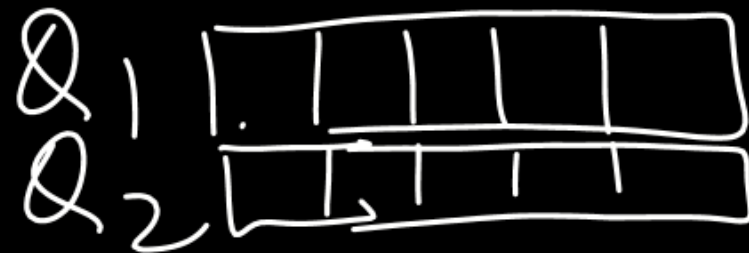
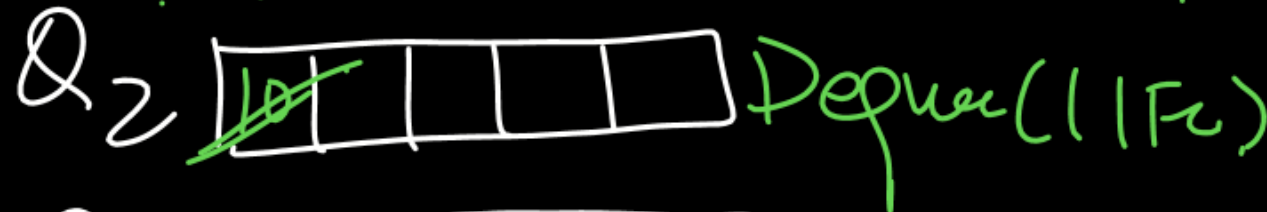
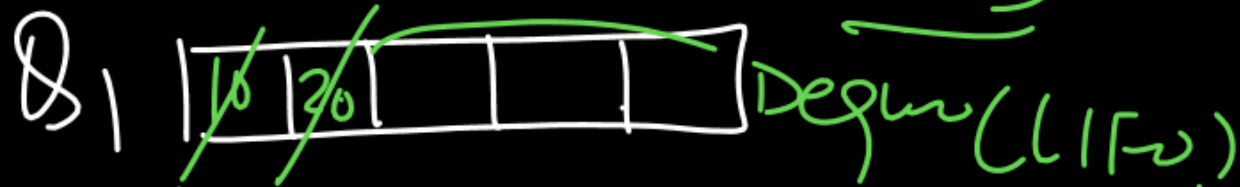
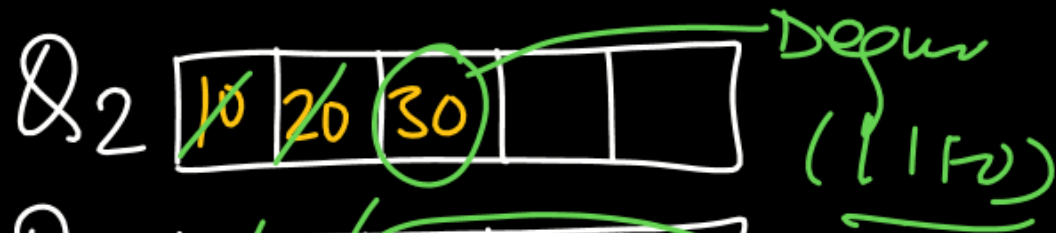
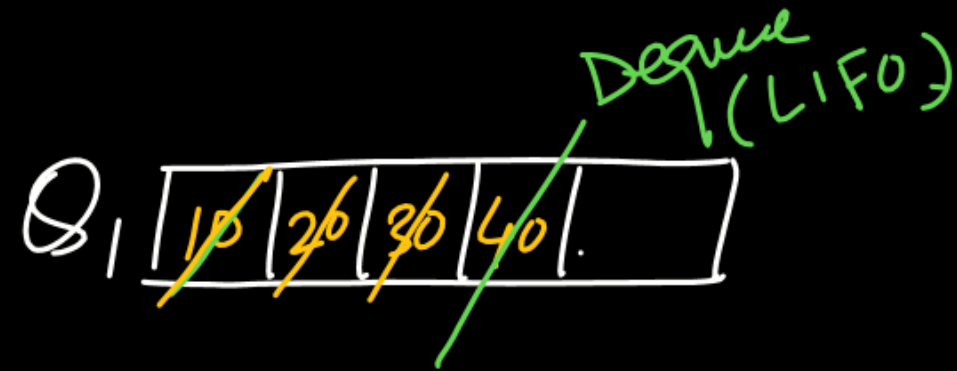
$$y_{\min} = 6 \quad | \quad y_{\max} = 8$$

Implementing a Stack using Queues:- 2 Queue

Costly Pop:



STACK.
(LIFO)



H.W

no. of Push = n

no. of Pop = m ($m \leq n$)

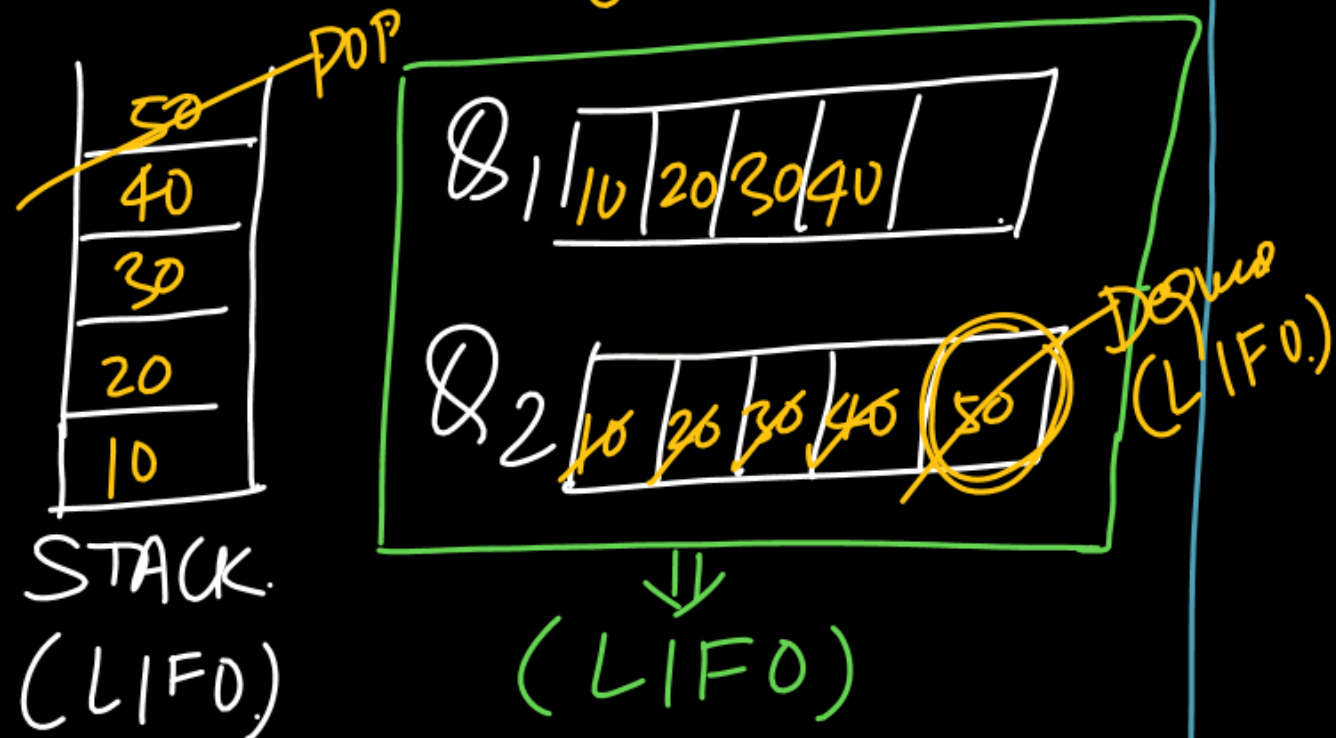
x = Enqueue operations

y = Dequeue operations

$|x|$ $\begin{cases} x_{\min} = ? \\ x_{\max} = ? \end{cases}$

$|y|$ $\begin{cases} y_{\min} = ? \\ y_{\max} = ? \end{cases}$

Implement Stack using Queues



Algo: Push(Q_1, Q_2, x)

{ if (Q_1 is Empty)

Enqueue(Q_2, x),

else

Enqueue(Q_1, x),

}

Algo: int Pop(Q_1, Q_2)

{ if (Q_1 is Empty)

{ if (Q_2 is Empty)

{ pf("Stack is Empty")

return -1

}

else { while (Q_2 does not contain one element)

{ x = Dequeue(Q_2),

Enqueue(Q_1, x),

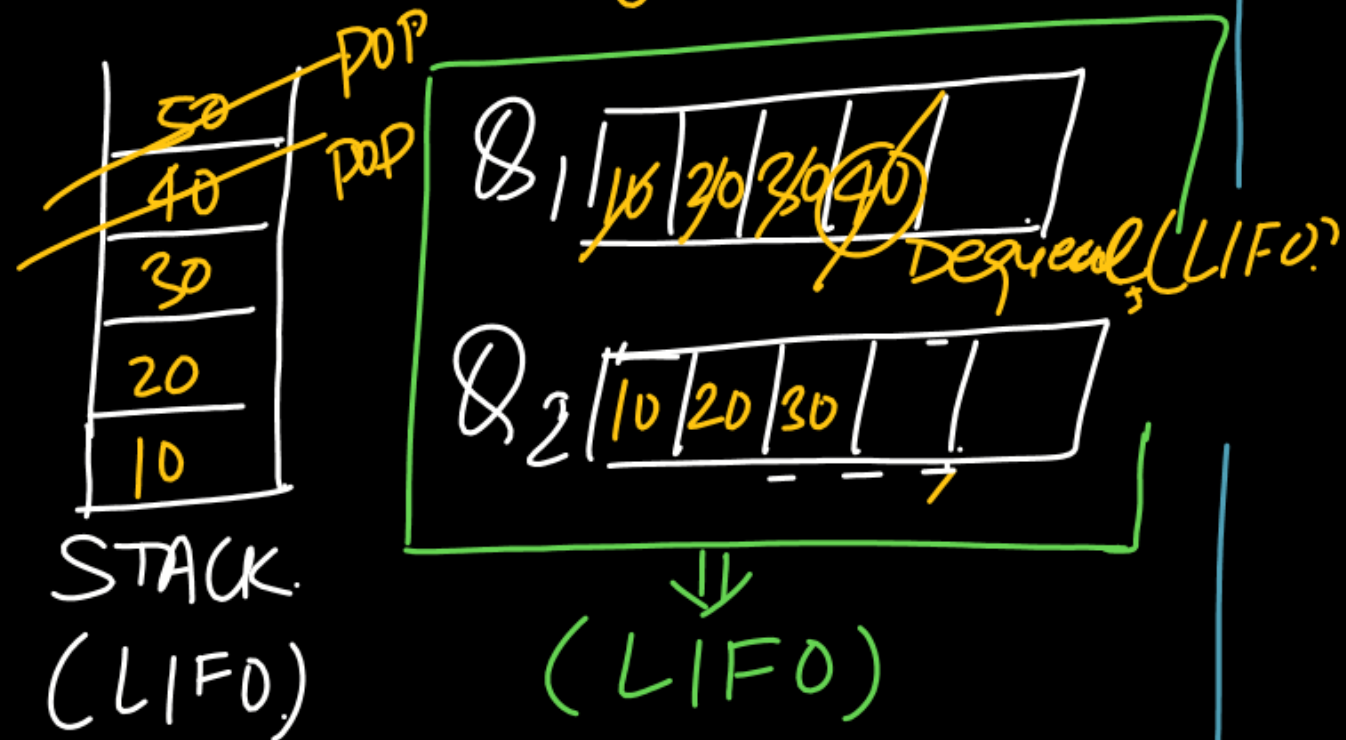
return Dequeue(Q_2),

}

}

}

Implement Stack using Queues



Algo: Push(Q_1, Q_2, x)

{ if (Q_1 is Empty)

Enqueue(Q_2, x);

else

Enqueue(Q_1, x);

}

Algo: int Pop(Q_1, Q_2)

{ if (Q_1 is Empty)

{ if (Q_2 is Empty)

{ pf("Stack is Empty")

return -1

else

{ while (Q_2 does not contain one element)

{ $x = \text{Dequeue}(Q_2)$;

Enqueue(Q_1, x);

return Dequeue(Q_2);

else

{ while (Q_1 does not contain one element)

{ $x = \text{Dequeue}(Q_1)$;

Enqueue(Q_2, x);

return Dequeue(Q_1);

}

Question:- Consider a Queue Q and Stack S initially empty

Push elements 10, 20, 30, 40, 50 into S, Q in the same order enqueue

H.W

1) POP(S)

2) Dequeue(Q)

3) Enqueue(Peek(S), Q)

4) Push(Dequeue(Q), S)

5) POP(S)

6) Enqueue(POP(S), Q)

7) Dequeue(Q)

8) POP(S)

9) Enqueue(Peek(S), Q)

10) Push(Dequeue(Q), S)

11) $x = \text{POP}(S)$

12) $y = Q[\text{front}]$

13) $z = Q[\text{rear}]$

14) $w = \text{Peek}(S)$

$$w + x + y - z = ?$$

Consider the following sequence of operations on an empty stack.

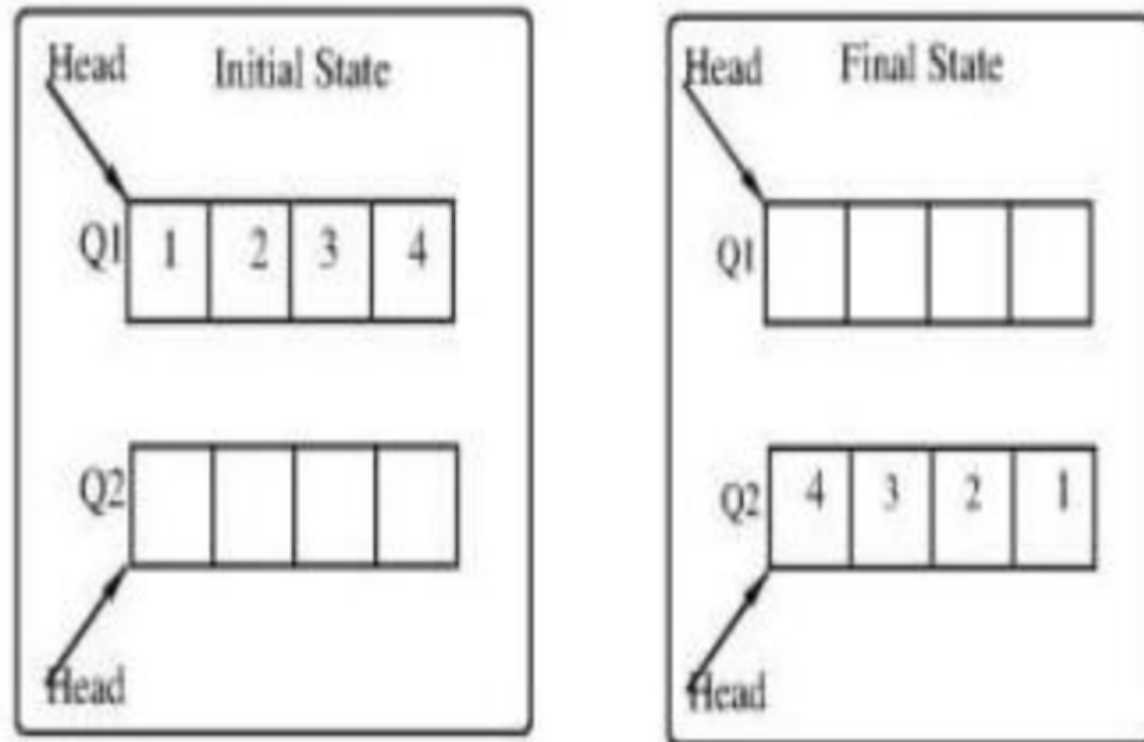
```
push(54); push(52); pop(); push(55); push(62); s=pop();
```

Consider the following sequence of operations on an empty queue.

```
enqueue(21); enqueue(24); dequeue(); enqueue(28); enqueue(32); q=dequeue();
```

The value of $s+q$ is _____.

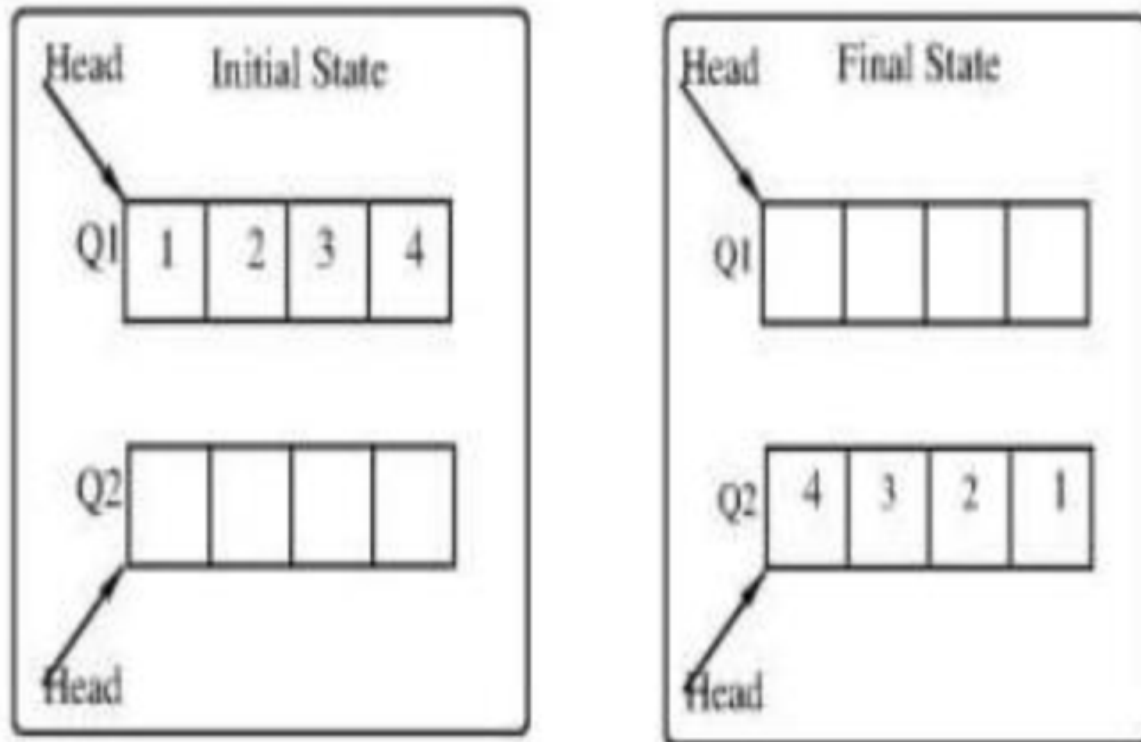
Consider the queues Q1 containing four elements and Q2 containing none (shown as the Initial State in the figure). The only operations allowed on these two queues are *Enqueue(Q, element)* and *Dequeue(Q)*. The minimum number of *Enqueue* operations on Q1 required to place the elements of Q1 in Q2 in reverse order (shown as the Final State in the figure) without using any additional storage is



Ans: 0

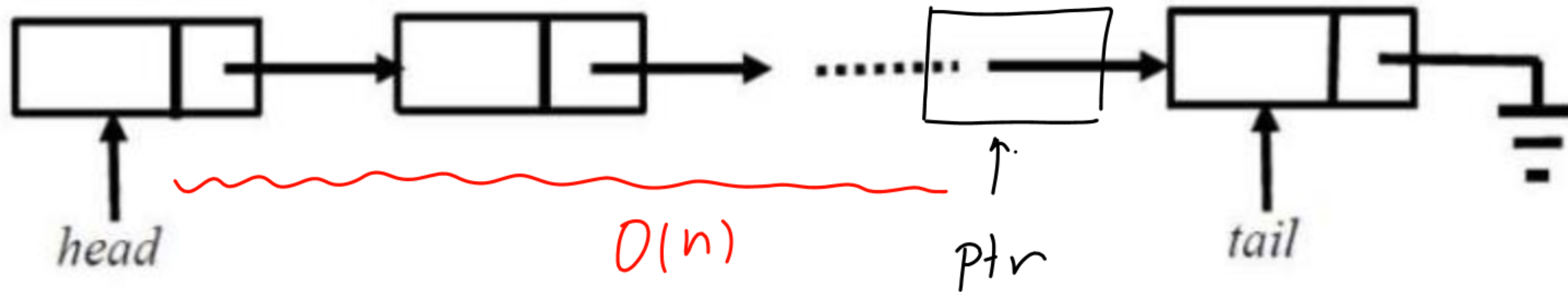
modified:

Consider the queues Q1 containing four elements and Q2 containing none (shown as the Initial State in the figure). The only operations allowed on these two queues are *Enqueue(Q, element)* and *Dequeue(Q)*. The minimum number of *Enqueue* operations on Q_2 required to place the elements of Q1 in Q2 in reverse order (shown as the Final State in the figure) without using any additional storage is



Ans: ?

A queue is implemented using a non-circular singly linked list. The queue has a head pointer and a tail pointer, as shown in the figure. Let n denote the number of nodes in the queue. Let enqueue be implemented by inserting a new node at the head, and dequeue be implemented by deletion of a node from the tail.



Which one of the following is the time complexity of the most time-efficient implementation of enqueue and dequeue, respectively, for this data structure?

- ☒ A $\Theta(1), \Theta(1)$
- ☐ B $\Theta(1), \Theta(n)$
- ☐ C $\Theta(n), \Theta(1)$
- ☐ D $\Theta(n), \Theta(n)$

Let Q denote a queue containing sixteen numbers and S be an empty stack. $\text{Head}(Q)$ returns the element at the head of the queue Q without removing it from Q . Similarly $\text{Top}(S)$ returns the element at the top of S without removing it from S . Consider the algorithm given below.

```

while  $Q$  is not Empty do
    if  $S$  is Empty OR  $\text{Top}(S) \leq \text{Head}(Q)$  then
         $x := \text{Dequeue}(Q)$ ;
        Push( $S, x$ );
    else
         $x := \text{Pop}(S)$ ;
        Enqueue( $Q, x$ );
    end
end
end

```

Ascending order.
 $Q, \boxed{1} \boxed{2}$

$Q, \boxed{} \boxed{}$

No of Iteration = (2)

STACK
 $\boxed{2}$
 $\boxed{1}$

The maximum possible number of iterations of the while loop in the algorithm is_____.

Let Q denote a queue containing sixteen numbers and S be an empty stack. $\text{Head}(Q)$ returns the element at the head of the queue Q without removing it from Q . Similarly $\text{Top}(S)$ returns the element at the top of S without removing it from S . Consider the algorithm given below.

```

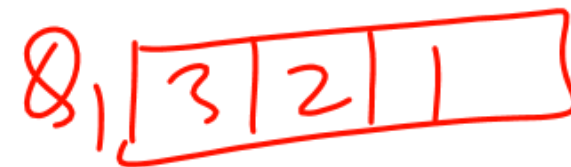
while  $Q$  is not Empty do
    if  $S$  is Empty OR  $\text{Top}(S) \leq \text{Head}(Q)$  then
         $x := \text{Dequeue}(Q)$ ;
        Push( $S, x$ );
    else
         $x := \text{Pop}(S)$ ;
        Enqueue( $Q, x$ );
    end
end
end

```

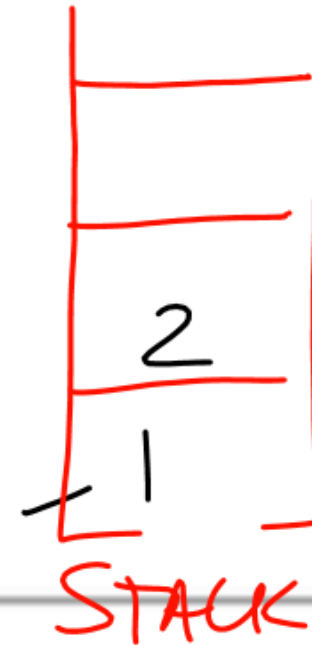
Descending order:



loop iteration: $1 + 1 + 1 + 1$
 $4 \text{ times} \Rightarrow 2 \times 2$

Similarly: 

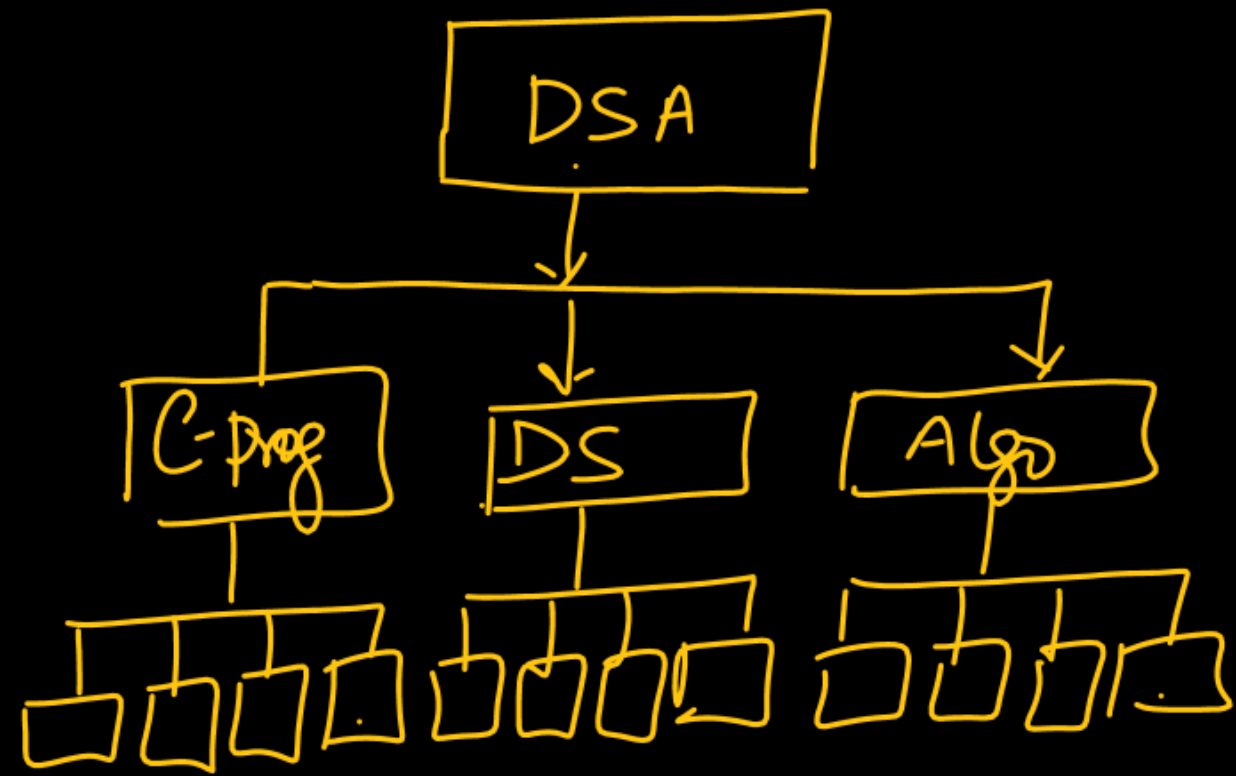
loop iteration $\Rightarrow (9) \Rightarrow 3 \times 3$
 $\Rightarrow n \times n$



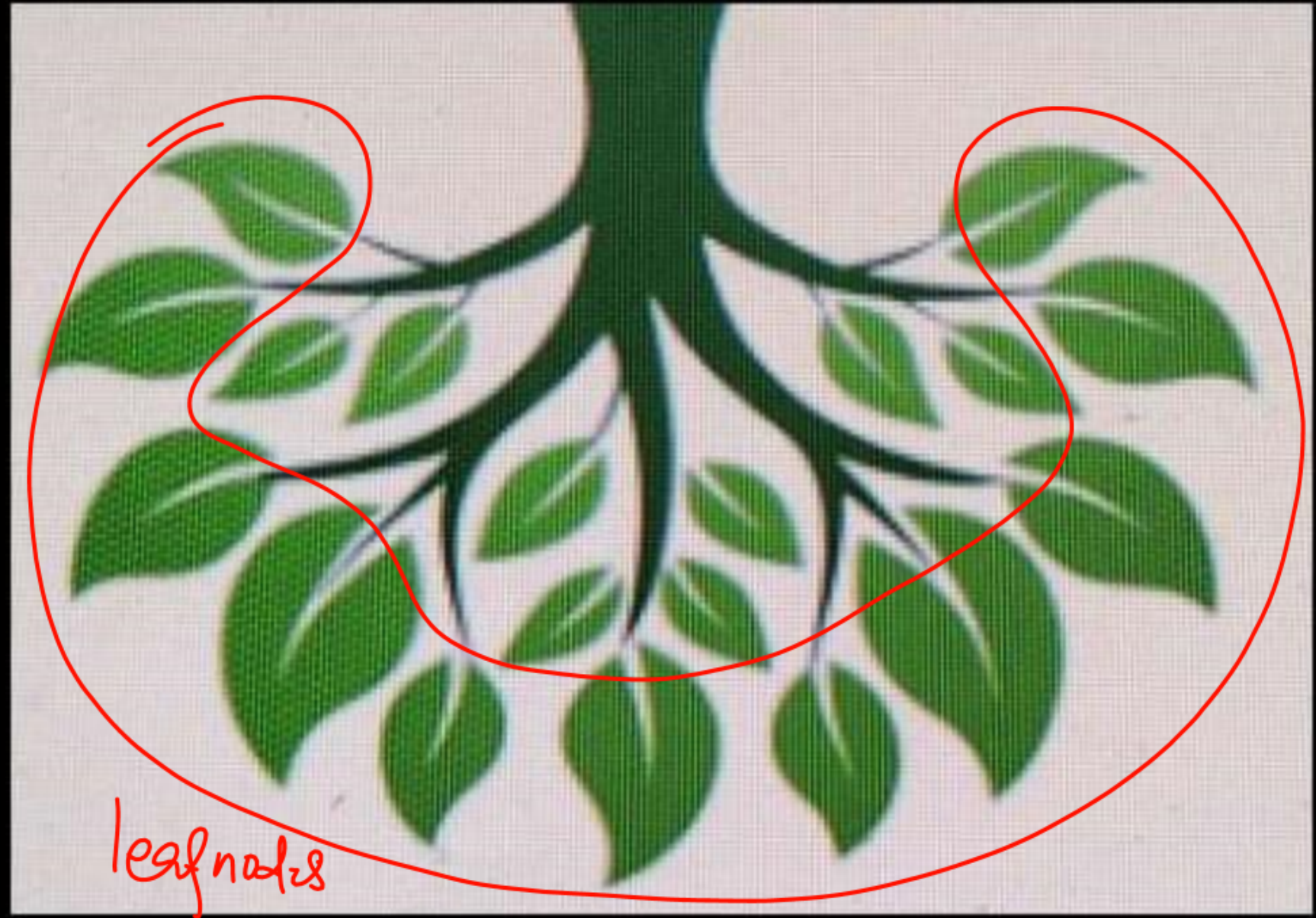
STACK.

The maximum possible number of iterations of the while loop in the algorithm is $16 \times 16 \Rightarrow (256)$ Ans

What Is a Tree?



Root:



Total nodes = Leaf nodes + Internal nodes

Tree :- Tree is a nonLinear data Structure used to store the data element by following Parents child relationship between the data elements

Level $\Rightarrow$ Depth

$h(\text{Tree}) = 3$

$h(A) = 3$
 $d(A) = 0$

Root of Tree

Level

1) Root: A

2) Leaf nodes: E L M N H O J P

3) Internal nodes: A B C D F G I K

4) Ancestors of O: I C A

5) Ancestors of P: K D A

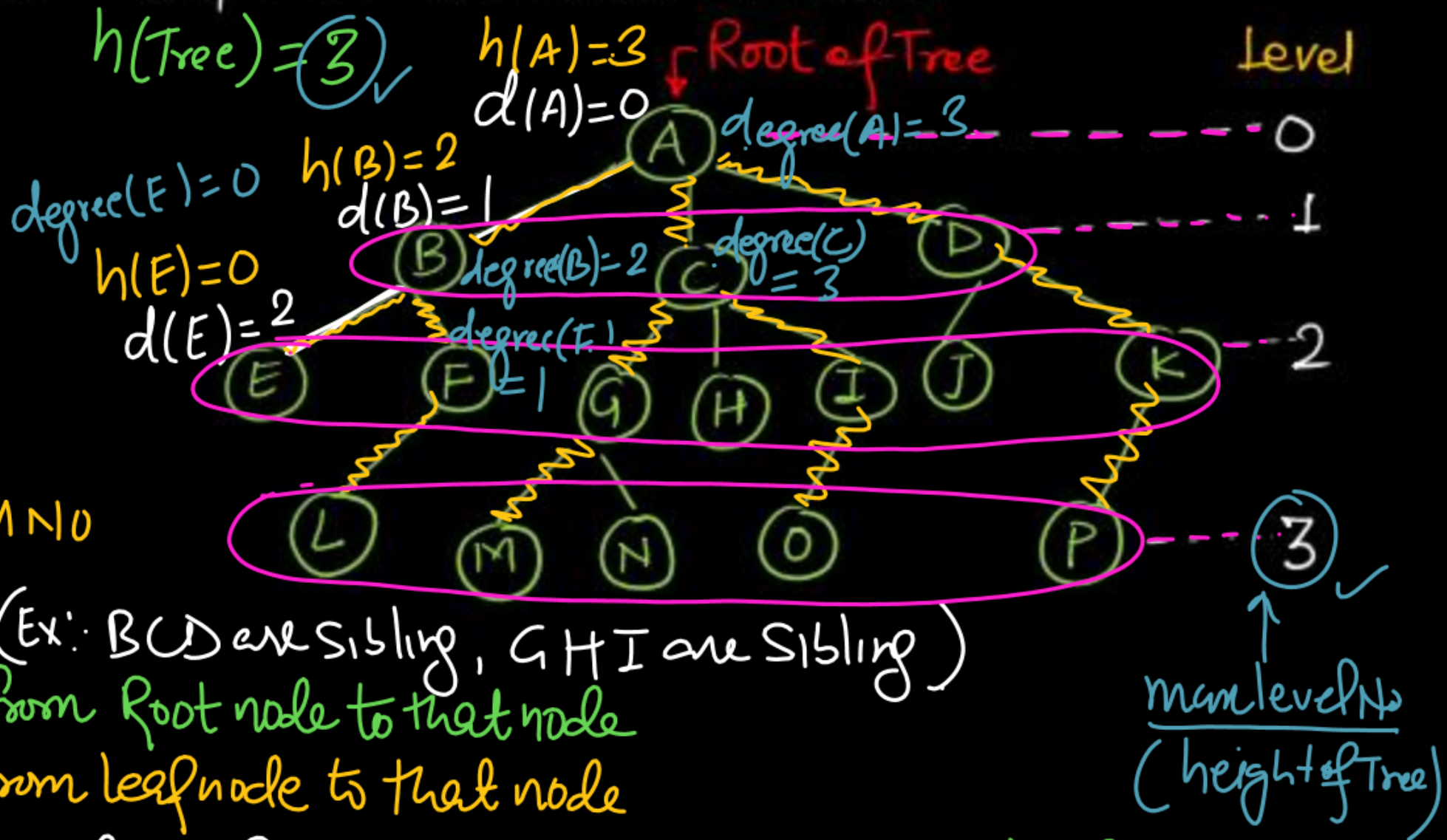
6) Descendant of C and A: G H I M N O

7) Sibling: Children of same Parent (Ex: B C D are sibling, G H I are sibling)

8) Depth of a Node: No. of Edges from Root node to that node

9) Height of a Node: ^{Max} No. of Edges from leaf node to that node

10) Height of the Tree: The longest Path from Root node to any leaf node. (Height of Root node)
(Maximum level No.)



Degree of tree :- Degree of a tree decides maximum No. of child nodes any node can contain
Degree of Leaf node is 0



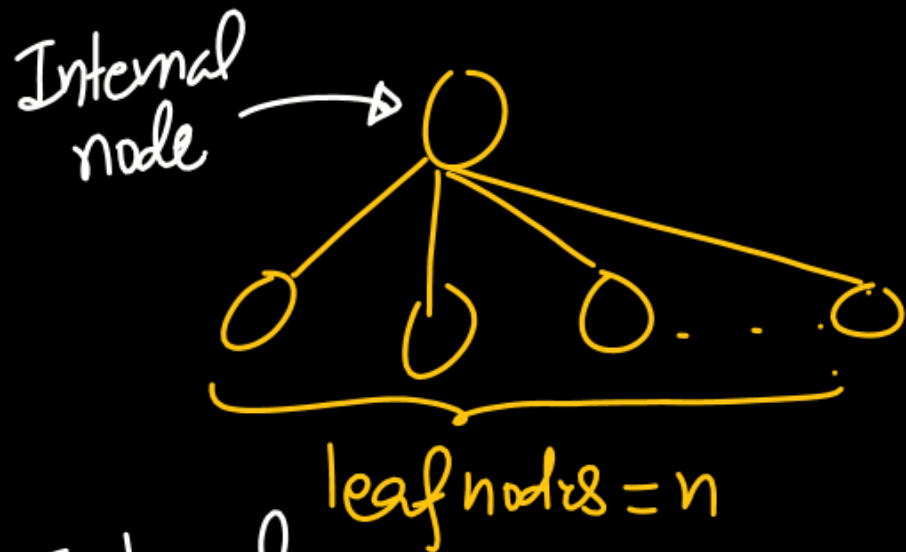
Height



Depth

n-ary Tree:- A rooted tree is called as n-ary tree if every internal vertex has no more than n-children. The tree is called a full n-ary tree if every internal vertex has exactly n-children. An n-ary tree with n=2 is called a Binary Tree.

Full n-ary Tree



$$L = i * (n-1) + 1$$

n-ary

$$N = L + i$$

$$N = i * n - i + 1 + i$$

$$N = i * n + 1$$

n-ary

Internal nodes

1

2

3

⋮

i

Leaf nodes

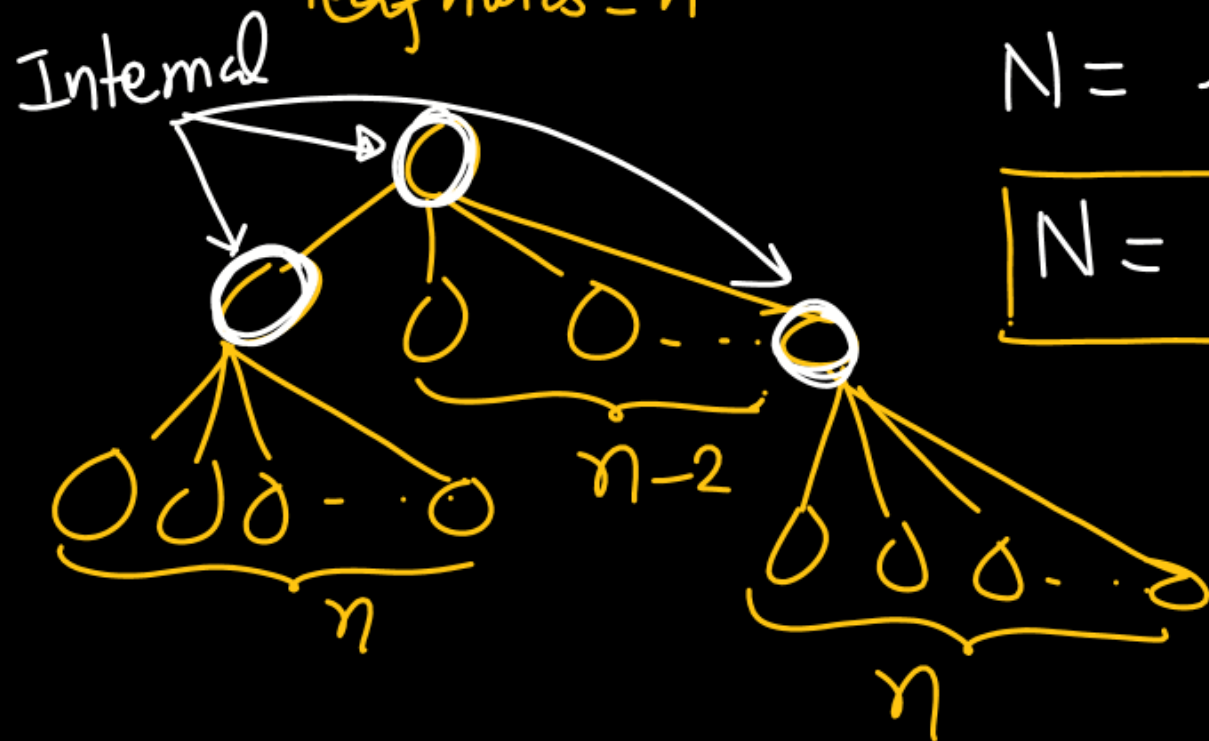
n

2n-1

3n-2

$i * n - (i-1)$

$i * (n-1) + 1$



Question:- A 3-ary tree is a tree in which every internal node has exactly 3 children
The no. of leaf nodes in such a tree with 6 internal node will be

a) 10

b) 23

c) 17

d) 13

$n = 3$ ary-Tree.

$$L = i * (n - 1) + 1$$

$$L = 6 * (3 - 1) + 1$$

$$L = 6 \times 2 + 1 = 13$$

Question:- A Complete n -ary tree is a tree in which each node has n -children or no children. Let I be the no. of internal node and L be the no. of Leaves in a Complete n -ary tree of $L=41$ and $I=10$, what is the value of n ?

a) 3

b) 4

✓ c) 5

d) 6

$$L = I * (n - 1) + 1$$

$$41 = 10 * (n - 1) + 1$$

$$n - 1 = \frac{41 - 1}{10} = 4$$

$$n = \textcircled{5} \text{ Ans}$$

Question:- The no. of Leafnodes in a rooted tree of n nodes with each node having 0 or 3 children is

a) $\frac{n}{2}$

b) $\frac{(n-1)}{3}$

c) $\frac{(n-1)}{2}$

☒ d) $\frac{(2n+1)}{3}$

$$L = i * (k-1) + 1$$

$$n = i * k + 1$$

$$n = i * 3 + 1$$

$$i = \left(\frac{n-1}{3}\right)$$

$$L = \left(\frac{n-1}{3}\right) * 2 + 1 \Rightarrow \frac{2n-2+3}{3} \Rightarrow \left(\frac{2n+1}{3}\right) \underline{\text{Ans}}$$

Q.13-Consider a binary tree T in which every node has either zero or two children. Let $n > 0$ be the number of nodes in T . Which ONE of the following is the number of nodes in T that have exactly two children? (GATE CSE 2025 Shift-II)

(A) $(n - 2) / 2$

$$k = 2$$

☒ (B) $(n - 1) / 2$

$$n = i * (k + 1)$$

(C) $n / 2$

$$l = \frac{n-1}{k}$$

(D) $(n + 1) / 2$

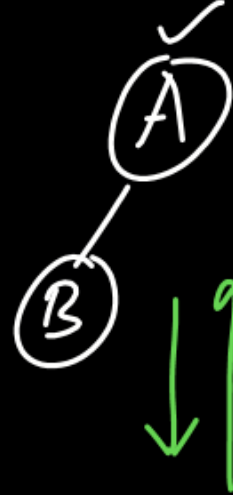
$$l = \left(\frac{n-1}{2} \right)$$

Binary Tree:-

Graphical.



(A, NULL, NULL)

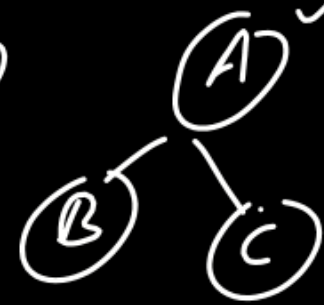


(A B NULL)

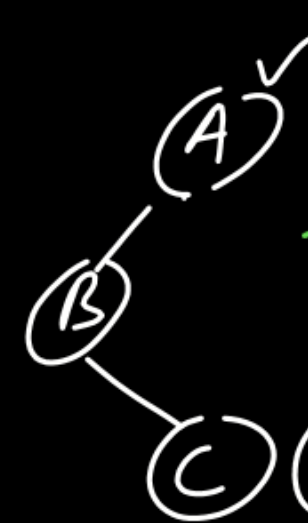
At most 2 children (0, 1, 2 children)



(A (B C NULL) NULL)



(A B C)



(A (B D E) (C F G))



Representation of Binary Tree:

1) Graphical Representation ✓

2) Nested Representation (Root left Right)

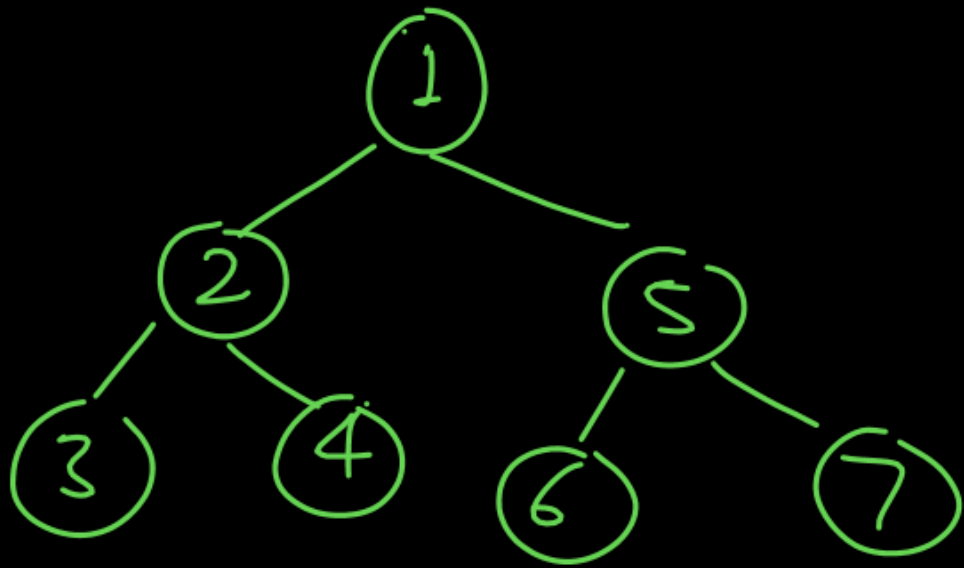
Ex:- Which of the following is valid Binary Tree

x a) (1 2 (4 5 6 7))

x b) (1 ((234)56)7)

✓ c) (1 (234) (567))

x d) (1 (2 3 NULL) (4 5))





Shailendra Singh GATE CSE/DA

THANKYOU



GeeksforGeeks GATE